5-4 · DataFrame과 SparkSQL 가공 · 정리·회고 0.5H

정리·회고

세 도구를 한 표로, 5-5 를 위해 파티션 수를 본다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① Pandas · PySpark API · SQL 문법을 한 표로 나란히 정리한다

② 테이블 셋과 파티션 수를 확인하고 compose 를 멈춘다

③ 자주 겪는 오류 여섯 가지의 원인과 해결을 안다

④ 5-5 파티셔닝·셔플링 최적화의 준비를 확인한다

설정은 200조각, sales 는 AQE 가 합쳐 1조각 — 이 차이가 5-5 의 출발점

오늘의 요약

세 도구 대조표 — 이름이 거의 그대로

하려는 일	Pandas	PySpark API	SQL
열 고르기	df[['a']]	df.select("a")	SELECT a
필터	df[df.a>1]	df.filter(F.col("a")>1)	WHERE a>1
파생	df.assign(c=...)	df.withColumn("c", ...)	SELECT ... AS c
집계	groupby('g').agg()	groupBy("g").agg()	GROUP BY g
조인	merge(b, on='k')	join(b, on="k")	JOIN ... ON
윈도	rolling / rank	F.row_number().over(w)	OVER (PARTITION BY ...)
정렬	sort_values('a')	orderBy("a")	ORDER BY a
저장	to_parquet()	writeTo(...).createOrReplace()	CREATE TABLE ... AS

헷갈리면 「Pandas 로는 뭐였지」를 먼저 떠올리세요 — 다른 점은 지연 평가와 F.col 정도

5-5 준비

다음 차시를 위한 준비 — 테이블과 파티션 수

1. **spark.sql("SHOW TABLES IN lake.db").show()**
   daily_revenue · product_sales · orders — 임시 뷰는 안 나온다

2. **sales.rdd.getNumPartitions()**
   sales 파티션: 1 — AQE 가 실행하며 빈 조각을 합쳤다

3. **spark.conf.get("spark.sql.shuffle.partitions")**
   기본 셔플 파티션: 200 — 이 값을 바꿔 보는 것이 5-5

파티션 수

sales 1

설정 200

수업을 마칠 때

지우지 않고 멈춘다 — exit() 에서 stop 까지

1. **exit()**
   >>> 에서 셸을 나온다 — 변수와 임시 뷰는 여기서 사라진다

2. **cd ~/data-eng/week5 · docker compose ps**
   컨테이너 넷이 Up

3. **ls**
   docker-compose.yml · first_job.py · lake-pyspark.sh

4. **docker compose stop**
   넷 Stopped — product_sales · daily_revenue 는 MinIO 볼륨에 남는다

> **5-5 는 docker compose start → bash lake-pyspark.sh 로 다시 시작합니다**

오늘의 체크리스트

열두 개를 스스로 확인해 보세요

#	확인한 것	#	확인한 것
1	Pandas 와 거의 같은 이름	7	윈도 함수로 날짜별 순위
2	API·SQL 이 같은 논리 계획	8	API·SQL 결과 일치 확인
3	F.col 로 컬럼 가리키기	9	explain() 두 방식 같은 구조
4	변환을 체인으로 파이프라인	10	SortMergeJoin 읽고 F.broadcast
5	조인 키 문자열 — 중복 없음	11	CTE 로 단계를 나눈 SQL
6	조인 전후 행 수·키 고유성	12	결과를 Iceberg 테이블로 저장

자주 겪는 오류

증상으로 찾는 원인과 해결 — 여섯 가지

증상	원인	해결
cannot resolve 컬럼명	오타 · 가공 전 DataFrame	직전 DataFrame printSchema()
product_id is ambiguous	양쪽에 같은 이름 컬럼	on="product_id" · alias
결과 없음 · …으로 잘림	action 없음 · show 기본값	show(truncate=False)
SHOW VIEWS AssertionError	Iceberg 확장이 가로챔	SHOW VIEWS IN default
NameError: 'orders'	셀 재시작 — 변수·뷰 소멸	1~3·6단계 다시 실행
조인 후 행 수가 늘어남	오른쪽 키에 중복	count · distinct().count()

먼저 「그 시점에 무엇이 있나」를 printSchema() · count() 로 보세요

예상 질문

자주 나오는 질문 다섯 가지

1. **API 와 SQL 중 뭘 배워야 하나요?**
   둘 다 — SQL 은 2주차 MySQL · 3주차 DuckDB · 4주차 Trino 에서도 쓰이니 투자 대비 효과가 크다

2. **임시 뷰와 Iceberg 테이블은 뭐가 다른가요?**
   임시 뷰는 세션이 끝나면 사라진다 — Iceberg 테이블은 MinIO 에 파일로 남는다

3. **F.col 없이 문자열로 써도 되나요?**
   간단한 조건은 된다 — 이름에 공백이 있거나 복잡한 식에서는 F.col 이 안전

4. **Pandas 코드를 그대로 옮길 수 있나요?**
   거의 — 인덱스가 없고, apply 같은 행 단위 함수는 느리니 내장 함수로

5. **products 는 세 줄인데 왜 Broadcast 가 아닌가요?**
   추정 크기로 고른다 — createDataFrame 표는 크기를 몰라 F.broadcast 나 Iceberg 로

심화 과제 (선택)

더 해 볼 것 넷 — 5-5 예습

01

날짜별 매출 두 방식

API · SQL

결과 대조

02

Spark UI 그래프

SQL 탭에서

Exchange 찾기

03

products 를 크게

spark.range 1만·10만

BHJ · 100만 SMJ

04

5-3 의 orders

item = name left

webcam 은 NULL

> **spark.range 표는 크기를 계산할 수 있다 — 1만·10만 행 BroadcastHashJoin, 100만 행 SortMergeJoin(실측)**

다음 회차 예고

5-5 · 파티셔닝과 셔플링 최적화

01
이론 1H
파티셔닝과
셔플링 원리

02
실습 1.5H
repartition·
coalesce 셔플

03
실습 1H
배치 스크립트로
데이터 마트 산출

04
정리 0.5H
정리·회고
산출물 점검

준비: 5-3~5-4 compose(스파크 클러스터 + MinIO) · lake.db.orders · product_sales — 배치 스크립트와 마트 테이블이 주차 산출물

5-4 · 정리·회고 0.5H 마무리

다음 회차는 5-5 파티셔닝과 셔플링

「왜 느린가」를 파티션과 셔플로 다루고, 지금까지의 가공을 배치 스크립트로 묶어 5주차 산출물 데이터 마트를 만듭니다

- ✓ 파티션 = 태스크 병렬성의 단위
- ✓ 좁은 변환·넓은 변환과 셔플 지점
- ✓ repartition 과 coalesce 의 차이
- ✓ Spark UI 의 Shuffle Read/Write

준비할 것

compose stop 상태 · 테이블 셋